Building a Progressive Web Application with Angular

A progressive Web application (PWA) uses modern Web capabilities to deliver an app-like experience to users. These apps meet certain requirements, are deployed to servers, are accessible through URLs, and indexed by search engines.

The key features of a progressive Web app are listed below.

- *Connectivity independent:* It is enhanced with service workers to work offline or on low quality networks.
- *Fresh:* It is always up-to-date thanks to the service worker update process.
- *Safe:* It is served via HTTPS to prevent snooping and to ensure content has not been tampered with.
- *Discoverable:* It is identifiable as an 'application' thanks to W3C manifests and service worker registration scope, allowing search engines to find them.
- *Installable:* It allows users to 'keep' the apps they find most useful on their home screen without the hassle of an app store.

There is a lot here, but it boils down to a few key points.

Offline support: Apps should be able to work offline, whether it is displaying a proper 'offline' message or caching app data for display purposes.

Web app manifest: An app manifest file should describe the resources the app needs. This includes the app's displayed name, icons, as well as splash screen.

Service worker: The service worker provides a programmatic way to cache app resources and API responses.

Why PWAs are hot in the Web app space

On the one hand, we have native apps that are undoubtedly fast and efficient in most cases. On the other hand, there are websites that are somewhat slow and with sketchy connectivity issues, and these only get worse.

The Accelerated Mobile Pages Project (AMP) spearheaded by Twitter and Google was launched in 2016 to solve these slow connection issues. PWAs work flawlessly in all the possible scenarios. With a good connection, there is never a problem. The problem is when there is no or a slow connection and we are greeted with an error page.

Clearly, this can become most annoying if we have a slow connection. The page will seem to be loading, yet all we see is a blank screen. We just wait, wait and wait but the page never seems to load. This is where a PWA comes to our rescue. The best part about it is that you get the best user experience possible on a slow connection, or even with no connectivity.

Things to consider while building a PWA

There are two major things to consider while building a progressive Web application.

App manifest: This is a JSON file that defines an app icon, how to launch the app (standalone, fullscreen, in the browser, etc) and provides any other related information. It's located in the root of the app. A link to this file is required on each page that has to be rendered. It is added in the head section of the HTML page:

```
<link rel="manifest" href="/manifest.
json">
```

Service worker: This is where most of the magic happens. It's nothing